What Does a Structural Honesty Linter Find That Others Miss? A Three-Tool Overlap Decomposition Across 25 Python Codebases

Bilal Syed Arfeen
BayyinahEnterprise

November 2026

Abstract

We decompose the findings of three static analysis tools -- furqan-lint v2.1.0 (structural honesty), bandit 1.9.4 (security), and ruff 0.16.9 (style and convention) -- across 25 open-source Python packages (2.3M lines, 5,252 files). Using file-and-line co-location (window of 2 lines), we classify each of furqan-lint's 24,933 findings as genuinely unique (no co-located finding from either other tool), overlapping with ruff only, overlapping with bandit only, or overlapping with both. 45.5% of furqan-lint's findings (11,338) are genuinely unique. Of these, automated context-based classification on a stratified sample of 585 yields 99.3% claim precision (the flagged pattern is present) and a relevance distribution of 35.3% safety-relevant, 57.3% code-health, and 7.4% intentional design. The genuinely novel rules -- kwargs passthrough (SI-005), hasattr masking (SI-007), empty except blocks (MR-005), resource leaks (RC-*), and try-except-continue (MR-007) -- have no equivalent in bandit or ruff and account for the bulk of the unique signal. We provide raw JSON from all three tools for independent verification.

Note on terminology. Finding means a pattern flagged by a tool's rule. Claim precision means the flagged pattern is syntactically present. Relevance is classified by automated keyword-proximity heuristic, not blind human raters.

---

1. Introduction

Static analysis tools for Python are well-established in three categories: style and convention (ruff, flake8, pylint), type checking (mypy, pyright), and security scanning (bandit, semgrep). A practitioner adding a new tool to their CI pipeline needs to know: does it find anything the existing tools miss?

furqan-lint is a multi-language static analysis tool organized around *structural honesty* -- whether code's interface claims match its implementation behavior. An earlier finding-count comparison (Arfeen, 2026) showed that furqan-lint produces more findings than bandit in 22 of 25 packages, but did not decompose the overlap or measure precision. This paper closes both gaps.

We ask three questions:

1. **Overlap.** How much of furqan-lint's output is already caught by bandit, ruff, or both?
2. **Precision.** When furqan-lint flags a pattern that neither other tool flags, is the pattern actually there?
3. **Relevance.** Of the genuinely unique true positives, how many matter for safety vs. code health vs. intentional design?

---

2. Method

2.1 Tools and configuration

Tool	Version	Configuration	Role
furqan-lint	v2.1.0	Default (39 active Python rules)	Structural honesty
bandit	1.9.4	Default (all rules)	Security scanning
ruff	0.16.9	30 rules mapped to furqan equivalents	Style + convention + security

ruff was run with a targeted rule set: the 30 ruff rules that have known furqan-lint equivalents (ANN401, BLE001, E722, FIX001/002, PGH001/002/003, S101, S104-S110, S301-S303, S307, S311, S324, S501, S602/603/605/606/608, TD001-003). This is a conservative test: if ruff catches a finding at the same location, the furqan-lint finding is not unique. Running ruff with all rules enabled would increase the overlap.

2.2 Corpus

25 packages from PyPI, scanned in installed form (site-packages). Domain diversity: web frameworks, HTTP clients, data science, ORMs, task queues, templating, cryptography, SSH, scraping, CLIs, cloud SDKs, image processing. Total: 2,327,868 lines, 5,252 files.

2.3 Overlap decomposition

For each furqan-lint finding, we check whether bandit or ruff produced a finding on the same file within 2 lines. A furqan-lint finding is classified as:

- **UNIQUE**: no bandit or ruff finding within 2 lines on the same file
- **RUFF_ONLY**: co-located with ruff but not bandit
- **BANDIT_ONLY**: co-located with bandit but not ruff
- **BOTH**: co-located with both

Co-location is a proxy for overlap, not proof of semantic equivalence. Two findings on the same line may detect unrelated patterns (coincidental co-location). The opposite also occurs: two tools may detect the same pattern on different lines (missed semantic overlap). We report both cases.

2.4 Precision study

From the UNIQUE pool (11,338 findings), we drew a stratified random sample: 50 per rule with 100+ unique findings, 20 per rule with 10-100, all for rules with fewer than 10. Total sample: 585 findings across all 33 rules with at least one unique finding.

For each sampled finding, we read the source code at the flagged location and answered two questions:

- **Q1 (claim precision):** Is the rule's stated pattern actually present in the code? (yes/no)
- **Q2 (relevance):** Does this instance matter? (SAFETY -- could cause incident; HEALTH -- degrades reliability/maintainability; DESIGN -- intentional choice matching pattern)

Rater. Classification was performed by an automated context heuristic (Claude Opus 4.6), not by human raters blind to rule ID. Q1 checks for contextual contradictions (e.g., a close() just outside the scan window). Q2 uses keyword proximity (e.g., "auth", "connect", "execute" near an empty except block signals safety relevance). Both should be verified by human raters for a definitive measurement.

2.5 Threats to validity

1. **Co-location is not equivalence.** Some co-located findings detect unrelated patterns; some semantic overlaps occur on different lines.
2. **Ruff rule set is conservative.** Only 30 of ruff's 800+ rules were tested. Full ruff coverage would increase overlap.

3. **Automated classification.** Q2 relevance uses keyword proximity, not human judgment. Safety rates may differ with human raters.
4. **Installed packages include test files.** Affects assert-related rules.
5. **Convenience sample** of packages.

---

3. Results

3.1 Three-way overlap

Category	Count	% of furqan	Description
UNIQUE	11,338	45.5%	No bandit or ruff finding within 2 lines
RUFF_ONLY	9,465	38.0%	ruff fires at same location, bandit does not
BANDIT_ONLY	264	1.1%	bandit fires at same location, ruff does not
BOTH	3,866	15.5%	Both bandit and ruff fire at same location

45.5% of furqan-lint's findings are genuinely unique -- neither bandit nor ruff produces a finding at the same location. The overlap is stable across line windows: 46.8% unique at window=0, 45.5% at 2, 43.7% at 5.

3.2 Per-rule uniqueness

Rules with the highest proportion of unique findings (n >= 20):

Rule	Total	Unique	% unique	Detection target
MR-007	100	97	97.0%	try-except-continue in loop
SI-003	269	262	97.4%	Blanket # noqa
RC-004	73	69	94.5%	Temp file not cleaned up
RC-001	85	78	91.8%	open() without close
SI-007	2,017	1,791	88.8%	hasattr() swallows AttributeError
MR-005	964	829	86.0%	Empty except block
SI-005	2,794	2,252	80.6%	**kwargs passthrough
SI-001	11,742	5,483	46.7%	Any type annotation

Rules with the lowest uniqueness:

Rule	Total	Unique	% unique	Covered by
SI-006	2,872	0	0.0%	bandit B101 (assert)
SI-004	1,894	40	2.1%	ruff TD/FIX (TODO)
SI-002	666	18	2.7%	ruff PGH003 (type: ignore)
TB-005	115	1	0.9%	bandit B307 (eval)
TB-006	153	3	2.0%	bandit B301 (pickle)

3.3 Claim precision (Q1)

Metric	Value	95% CI
Sample size	585	

Metric	Value	95% CI
True positives	581	
False positives	4	
Precision	**99.3%**	**[98.3%, 99.7%]**

The 4 false positives: 3 RC-001 (a close() existed outside the scan window in a try/finally or contextlib.closing) and 1 TB-018 (log message with SQL-like keywords).

3.4 Relevance (Q2, true positives only)

Category	Count	%	95% CI	Description
SAFETY	205	35.3%	[31.5%, 39.3%]	Could cause security breach, data loss, or outage
HEALTH	333	57.3%	[53.3%, 61.3%]	Degrades reliability, maintainability, debuggability
DESIGN	43	7.4%	[5.5%, 9.8%]	Intentional design choice matching the pattern

3.5 Relevance by rule family

Rule	n	SAFETY	HEALTH	DESIGN	Primary signal
TB-* (trust boundary)	104	104	0	0	Security: mutable defaults, SQL injection, crypto, buffer overflow
RC-* (resource close)	96	58	38	0	Safety when sockets/connections; health for files
MR-* (error handling)	101	13	88	0	Safety when auth/connection; health otherwise
SI-005 (kwargs)	50	1	6	43	Mostly intentional design (86%)
SI-001 (Any)	50	0	50	0	Type quality only
SI-007 (hasattr)	50	1	49	0	Code health, rarely safety
SI-003 (noqa)	50	0	50	0	Suppression hygiene
SI-004 (TODO)	20	0	20	0	Completeness signal
MR-007 (continue)	20	0	20	0	Loop error visibility

---

4. Discussion

4.1 The novel signal

furqan-lint's genuinely unique contribution -- the 45.5% that neither bandit nor ruff catches -- comes from five rule families:

Error handling (MR-005, MR-007, MR-001, MR-002, MR-003): 1,007 unique findings. Empty except blocks, bare except without re-raise, try-except-continue. These are the structural honesty core: code that claims to handle errors while silencing them. 13% are safety-relevant (swallowed auth/connection exceptions).

Resource lifecycle (RC-001 to RC-005): 258 unique findings, 91.8% precision. Open without close, acquire without release, unmanaged connections. 60% safety-relevant (socket and connection leaks).

Interface honesty (SI-005): 2,252 unique findings (80.6% unique rate). **kwargs accepted and forwarded without validation. However, 86% are intentional design choices in plugin systems and wrappers. The 14% health-relevant cases are functions where the unbounded interface masks API errors.

Defensive masking (SI-007): 1,791 unique findings (88.8% unique rate). hasattr() swallowing AttributeError. Almost entirely code health (98%).

Trust boundary (TB-004, TB-018, TB-023-028): Various unique security findings with 100% safety classification but small counts.

4.2 What is not novel

54.5% of furqan-lint's output overlaps with bandit or ruff at the same location:

- **SI-001 (Any annotation):** 53.3% overlaps with ruff ANN401. The 46.7% unique portion may reflect ruff version differences or line-offset mismatches, not genuinely different detection.
- **SI-006 (assert):** 100% overlaps with bandit B101. Zero unique value.
- **SI-004 (TODO):** 97.9% overlaps with ruff TD/FIX.
- **TB-005, TB-006 (eval, pickle):** Almost entirely covered by bandit.

4.3 The case for complementarity

furqan-lint's unique signal is concentrated in error handling and resource lifecycle -- patterns that fall between security (bandit) and style (ruff). These are not CVEs, not formatting issues, and not type errors. They are structural gaps where code's interface claims diverge from its behavior.

The strongest evidence: RC- *and MR*- rules have no equivalent in either bandit or ruff, 86-97% uniqueness rates, 99%+ claim precision, and a mix of safety and health relevance.

The weakest evidence: SI-001 (47% of all findings, 47% unique) is a type-quality check with a direct ruff counterpart. SI-005 (80.6% unique) is 86% intentional design. The tool's output is dominated by rules that are either already covered or acceptably noisy.

4.4 Limitations

1. **Automated rater.** Q2 relevance was classified by keyword proximity, not human raters blind to rule ID. Safety rates should be verified by human review.
2. **Conservative ruff coverage.** Only 30 of ruff's rules were tested. Full ruff may increase overlap.
3. **Co-location proxy.** File+line matching misses semantic overlaps on different lines and counts coincidental co-location as overlap.
4. **Single tool versions.** Results may differ with other versions.
5. **Installed packages include test files.**
6. **SI-005 design rate.** If 86% of kwargs findings are intentional, the effective unique-and-relevant count drops by ~1,900.

---

5. Conclusion

45.5% of furqan-lint's findings are genuinely unique -- not caught by bandit or ruff at the same location. The unique signal has 99.3% claim precision and is 35.3% safety-relevant, 57.3% health-relevant, and 7.4% intentional design.

The genuinely novel rules -- error handling (MR-*), resource lifecycle (RC-*), hasattr masking (SI-007), and to a lesser extent kwargs passthrough (SI-005) -- have no equivalent in either security or style linters. They detect patterns that fall in the gap between "known vulnerability" and "formatting issue." Whether this gap constitutes a distinct analysis class is a naming decision, not an empirical one. The empirical finding is that 11,338 findings across 2.3M lines of production code go undetected by the two most widely-used tools in their respective categories.

The ~4,000 safety-relevant unique findings (extrapolated from 35.3% of 11,338) represent patterns that could contribute to incidents in production and that no other tool in the comparison flags. This is the practical case for adding furqan-lint alongside bandit and ruff in a CI pipeline.

Open questions for future work:

- Human rater verification of the 35.3% safety rate
- Full ruff rule set comparison (800+ rules)
- semgrep inclusion as a fourth tool
- Longitudinal study: do furqan-lint findings predict actual production incidents?

Data availability

All raw data: benchmark_v2.1.0.csv, raw_json/ (furqan-lint, 24,933), bandit_json/ (bandit, 29,959), ruff_json/ (ruff, 44,855), overlap/ (two-way and three-way decomposition CSVs), precision_v2/ (585 classified findings with source context), versions.json, rule_manifest.json. Tools: furqan-lint v2.1.0, bandit 1.9.4, ruff 0.16.9, Python 3.13.18. Repository: https://github.com/BayyinahEnterprise/Kun-Fayakun

---

References

[1] B. S. Arfeen, "furqan-lint vs. bandit: A Finding-Count Comparison Across 25 Python Codebases," BayyinahEnterprise, 2026.

[2] PyCQA, "Bandit: Security linter for Python," v1.9.4, https://bandit.readthedocs.io/

[3] Astral Software, "Ruff: An extremely fast Python linter," v0.16.9, https://docs.astral.sh/ruff/

[4] M. Brunsfeld et al., "Tree-sitter," https://tree-sitter.github.io/tree-sitter/

[5] OASIS, "SARIF Version 2.1.0," 2020.
